



PHP form validation 60m

When you complete a customer registration page, and you don't enter the data in the format that the form expects – for example, you don't enter a valid email address or fill out a mandatory field – then you'll get an error message. This is known as form validation. The data you enter into the form is being validated to ensure that it is in the correct format and falls within the constraints set by the web application.

Form validation is important because it protects your form from hackers. It keeps your data clean, correct and useful.

As we have alluded to in earlier topics, there are two areas where validation is possible: client-side validation and server-side validation.

- **Client-side validation** is performed on the web browser before the data is sent to the server. It works to alert the user with immediate feedback if the data they've inputted is incorrect. The data originating from the client side should not be trusted to be valid, as the server side has no control over how the client side is used. We use the client side to perform data cleansing to mitigate possible issues. This also reduces the number of return errors/responses to the client side.
- **Server-side validation** is performed on the server after the data has been validated and checked on the client side and then sent to the server. Server-side validation ensures the data submitted to the server is consistent with the constraints set by the web application. Server-side validation **MUST** always be performed regardless of what is prepared on the client side. We have no control over how the client side is used or whether the data from the client side has been compromised, so we need to ensure the data is clean and valid.

[Skip to main content](#)

Below you will find an example of PHP form validation. This example demonstrates how form validation can be used to alert users with error messages or warnings if their input is not in the correct format or does not conform to the constraints.

We will break this code down in the next section so you can understand how to use PHP form validation, but before we do, save this PHP code in the `www/projects` folder as `formregister.php` on your web server. Execute this code so you can understand how form validation works and how it alerts the user to their invalid entries and inputs.

Take notice of what happens when you do not fill in a required field, enter an email address without an `@` symbol or enter numbers instead of letters for the name field.

IMPORTANT: iQualify has an issue with the code `<?php` so we've had to put a space between the `?` and `php`. **Make sure you remove this space when you execute the code yourself.** There are comments, which indicate each section of the code where a space has been added between the `?` and `php`.



Copy

[Skip to main content](#)

```
1  <!DOCTYPE HTML>
2  <html>
3  <head>
4  <style>
5  .error {color: #FF0000;}
6  </style>
7  </head>
8  <body>
9
10 <? php
11 // define variables and set to empty values
12 $nameErr = $emailErr = $genderErr = $websiteErr = "";
13 $name = $email = $gender = $comment = $website = "";
14
15 if ($_SERVER["REQUEST_METHOD"] == "POST") {
16     if (empty($_POST["name"])) {
17         $nameErr = "Name is required";
18     } else {
19         $name = test_input($_POST["name"]);
20         // check if name only contains letters and whitespace
21         if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
22             $nameErr = "Only letters and white space allowed";
23         }
24     }
25
26     if (empty($_POST["email"])) {
27         $emailErr = "Email is required";
```



Explanation of PHP form validation code

Checking form submission

Notice that the script starts with the code:



[Skip to main content](#)



Copy

```
1  if ($_SERVER["REQUEST_METHOD"] == "POST")
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)

We discussed this on the last page, but this code is used to check whether the form has been submitted (that is, if the submit button has been pressed). If the form has been submitted, the form can be validated as per the conditions set out in the script for each field (for example, the gender field has been defined, so if the field is empty at submission, an error message is displayed to say 'Gender is required'; otherwise, if the field is not empty, the gender inputted is posted to the server.

Sanitizing data



Copy

```
1  function test_input($data) {  
2  $data = trim($data);  
3  $data = stripslashes($data);  
4  $data = htmlspecialchars($data);  
5  return $data;
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)

[Skip to main content](#)

The script above is used to sanitise the data so that the data being submitted through the form can do no harm to your website or server. The sanitising process does NOT perform the data validation. Sanitising means the data is cleaned, but no checks have been made to determine the context or datatype.

- The `trim()` function is used to strip unnecessary characters from a user's inputted data. The unnecessary characters can be things such as extra spaces, tabs or new lines .
- The `stripslashes()` function is used to remove backslashes from user inputs.
- The `htmlspecialchars()` function is an important function as it converts special HTML characters like `<` and `>` to `<` and `>`, preventing attackers from injecting malicious HTML or JavaScript code.

This means that if a user tried to hack a form by inputting the script

```
<script>location.href('http://www.youhavebeenhacked.com')
</script>
```

into a text field, instead of the hacked script working to change the form's original script, it would instead be converted to

```
<script>location.href('http://www.youhavebeenhacked.com')
</script>,
```

causing no problems to the form or script.

To avoid having to write out these three functions over and over again, we used the `test_input()` function, because this will do the checking for us without us needing to write the code multiple times.

Required fields

PHP can be used to define specific input fields as required, so they need to be filled before the form can be submitted. If the field is not filled, PHP can also create an error message.

You can see from the script that the fields 'Name', 'Email' and 'Gender' are required, so these fields cannot be empty; otherwise, the form will not submit.

Let's look at the name field:



Copy

```
1  if (empty($_POST["name"])) {  
2    $nameErr = "Name is required";  
3  } else {  
4    $name = test_input($_POST["name"]);  
5  }
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)



The variable `$nameErr` has been created for the error message. This script also has an `if...else` statement for the `$_POST` name variable, which checks if the name field is empty (using the `empty()` function) and if it is empty, the error message is stored in the `$nameErr` variable to be displayed by the HTML form script:



Copy

```
1  Email: <input type="text" name="email">  
2  <span class="error">* <?php echo $emailErr;?></span>
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)



[Skip to main content](#)

This code will generate the correct error message 'Name is required' if needed (that is, the name field is empty).



If the field is not empty, the data the user inputted into the name field will be sent through the `test_input()` function. Another thing that is not present in the above example is checking to see if the PHP variable even exists, never mind if it is empty. If a variable is not sent in the POST or GET, then it won't be available on the server side. This means you need to first use the `isset()` function to check if the variable exists before checking if it is empty or contains a value.

Checking sizes

One of the next most important validation steps is to check the size of the incoming data. In the case of the POST and GET variables, they are all strings until you convert them to their appropriate data type. For example, a postal code would come in as a series of digits, but in the form of string.

Checking the actual size or length of the data is important to ensure we only accept data of a certain length. Even though we can submit 1MB of text through an HTML form, it does not mean we should be receiving 1MB of data – a postal code field may only need 4 digits. Part of the size checking can be achieved through the database by using constraints. However, it is better to deal with the data before it reaches the database.

The `strlen()` function is a suitable choice for checking data sizes. In the case of an array, the `count()` function would work here too. You might read in some references that `sizeof()` is an alias for `count()`. However, do not use the `sizeof()` function, as its use is not synonymous with other programming languages that have the same named function but with a different functionality.

Validating fields

You can use PHP to check whether the data inputted into a field is in the correct format or contains the correct data type. For example, if you have a field that should only contain text, you can use PHP to accept only data that contains letters and whitespace. Or if you

[Skip to main content](#)

have an email address field, you'd expect the inputted data to contain an @ symbol and a dot (.) before the domain name (e.g., com), so you could use PHP to only accept data that contains both of these.

The script below checks if an email address is in a valid format. It uses the `filter_var()` function, which is used to validate and filter the email data because it has come from an insecure source (user inputs). We will learn about PHP filters next. At this stage, you just need to know that you can use the `filter_var()` function to validate an email address.

It is important to always check that the variable:

- exists
- is not empty (in some cases)
- is the right size or length,
- contains the right type of data (datatype and context).



Copy

```
1 // check if email address is well-formed
2 if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
3     $emailErr = "Invalid email format";
4 }
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)



The script below checks if the name contains only letters and whitespace.



[Skip to main content](#)



```
1 // check if name only contains letters and whitespace
2 if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
3     $nameErr = "Only letters and white space allowed";
4 }
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)



[Skip to main content](#)

☒ PHP forms

Non-Assessed

In this activity, you are going to modify the PHP form validation script we looked at in this section. What you need to do is:

- Create a field where users can select their hobbies. This will need to be a list of checkboxes that allows users to select one or more hobbies. This field is not required, so it does not need an error message.
- Create a field where users can input their phone number. This will need to be a required field that only accepts data in the form of numbers (including leading zeroes, +, spaces and brackets). So this field will require an error message if the field is empty and/or if the field contains anything other than numbers and the symbols listed.

☐ I have completed the task

Submit



Thoughts on this page?

